

Autonomous Production Engineer

Solvan is a governed fleet of institutional agents that detects production degradation from events, investigates live evidence, executes bounded policy-authorized recovery, verifies the outcome independently, and owns a multi-day permanent-repair case — with every step durable, fenced, and receipted in Cloud SQL.

Architecture stance. Two planes with a hard boundary: the *probabilistic plane* (Gemini agents on Agent Runtime) investigates, ranks, and proposes; the *deterministic plane* (typed services over Cloud SQL) owns permissions, state transitions, budgets, approvals, idempotency, mutations, and verification. Model output can become a proposal — never authority.

Competition alignment. Sections 14–15 assess Solvan through the lens of Google's own training lab for this challenge — *Build Long-Running Agents with ADK 2* (adk2003), "the ADK lab" throughout — mapping every mandated technology and every taught pattern to the enforcing mechanism and its named test. The lens demonstration runs on the release-gated payments slice; the submitted codebase goes further — most of the enterprise expansion is already implemented and locally verified beyond the lens, and the remainder is held as reviewed target contracts, each labelled as exactly what it is. Solvan is built for this competition and engineered to outlive it as Ruhu Inc.'s production reliability platform.

Python 3.12

Google ADK 2

Gemini on Vertex AI

Agent Runtime

PostgreSQL 16 · Cloud SQL

Cloud Run

Pub/Sub · Cloud Scheduler

FastAPI

React · TypeScript

OpenTelemetry · Cloud Trace

23

Deployable applications: coordinator, actuator, verifier, detector, trigger scheduler, evidence broker, five channel liaisons, relay control, workspace services, console, and fixtures.

260

PostgreSQL tables: the 61-table release schema plus eight implemented-or-specified target schemas — liaison, operability, alert triage, outcome quality, production graph, connection scope, relay, SaaS scale.

953

Test functions behind a ten-stage merge gate with PR-001..050 requirement-to-test traceability enforced in CI.

22

Normative specifications (17,218 lines) with executable artifacts: DDL, concurrency SQL, state-machine YAML, and release fixtures.

Revision

14 August 2026 · repo fba2365

Source

Local repository audit; claims link to tests, DDL, or specs

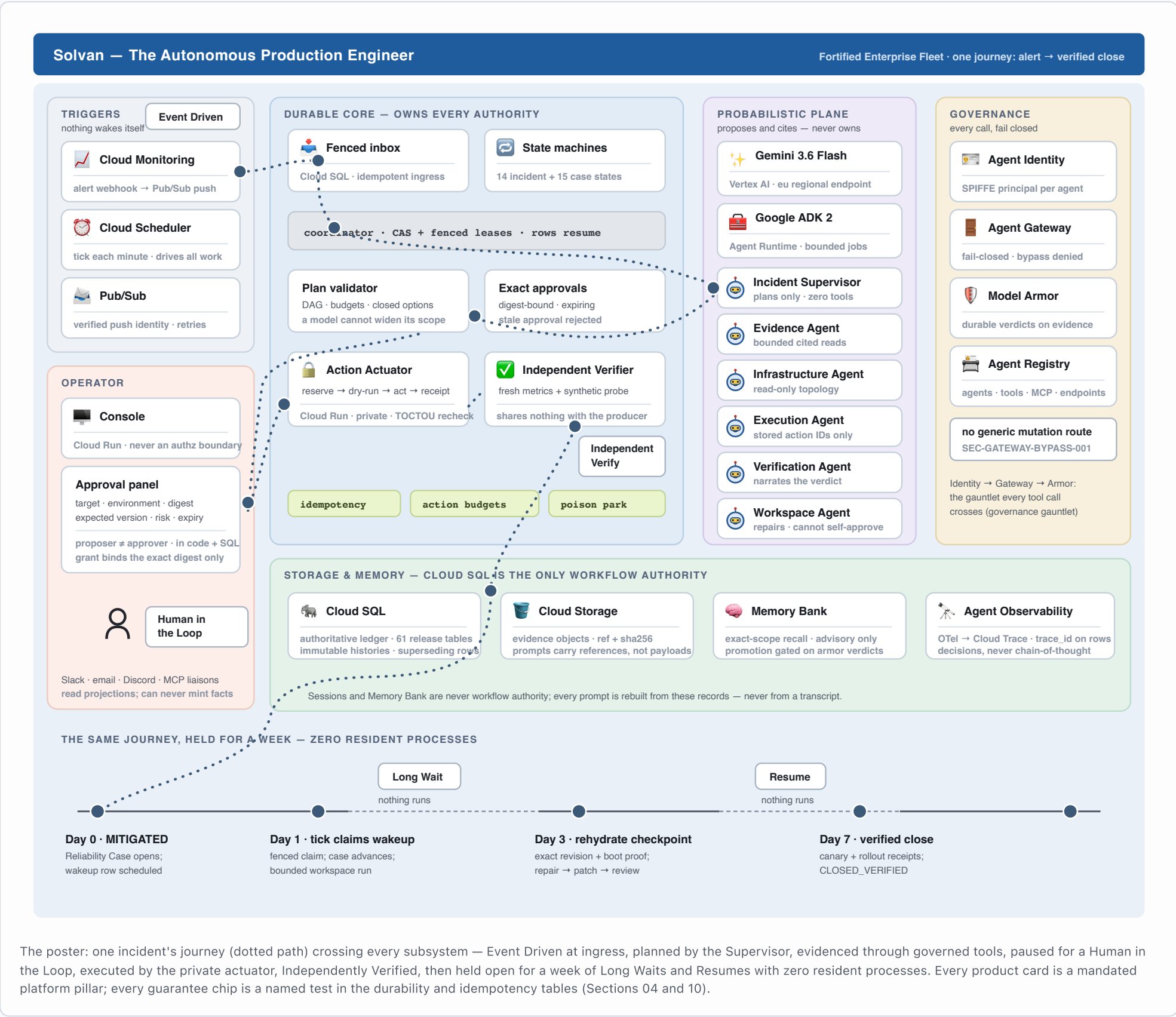
Print setup

A3 portrait, 14mm top/bottom, 18mm left/right

Scope

Runtime, agents, action safety, data, competition proof, deployment

01 The Solvan Poster: Executive Architecture



Runtime Contract

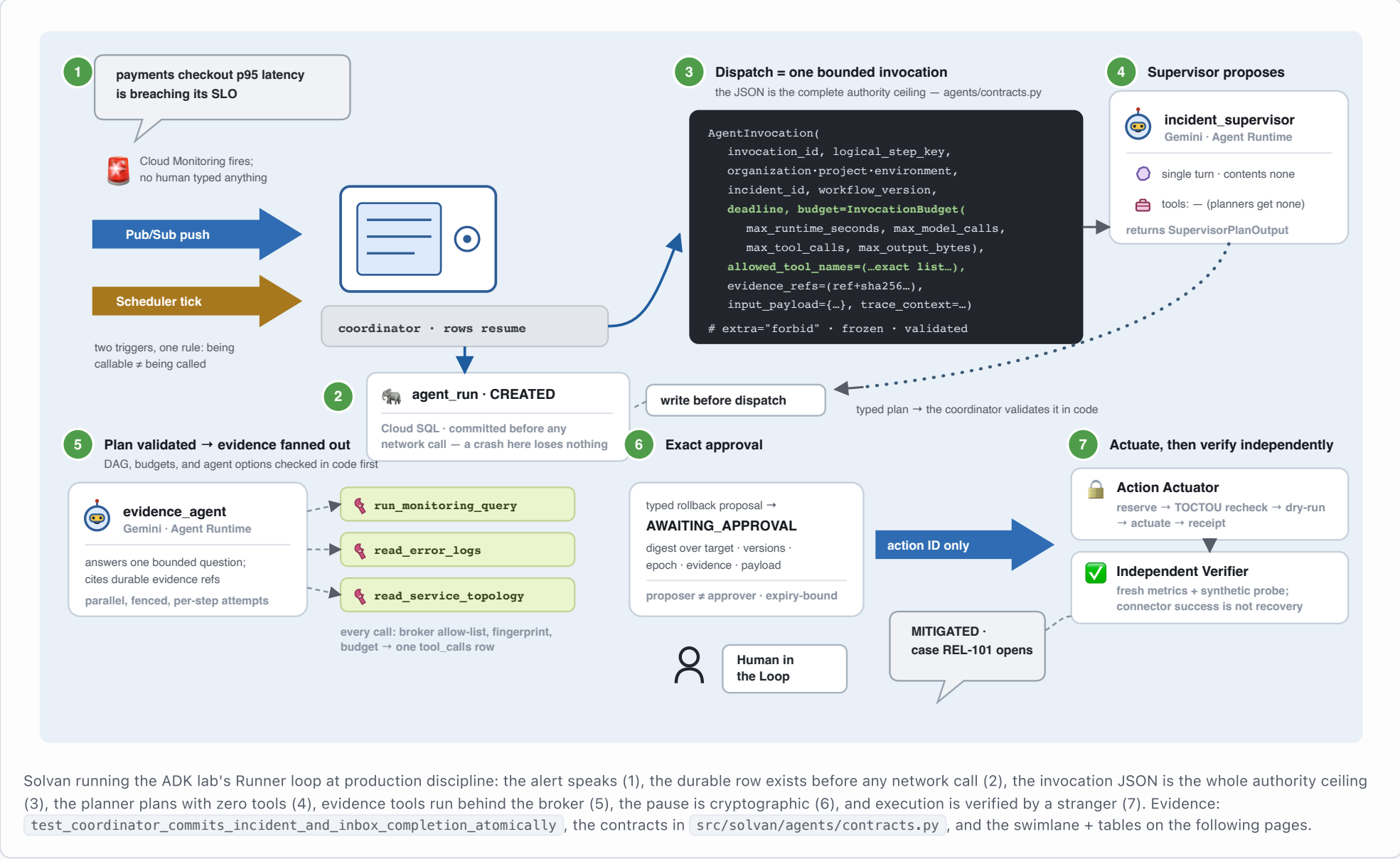
- Incidents begin from events, not prompts; chat reads projections and requests work but never defines the lifecycle.
- The coordinator creates every durable `agent_run` before dispatch; no agent invokes another agent directly.
- Every transition uses expected-state + workflow-version CAS under a token-fenced lease; terminal histories are immutable.
- Gemini investigates, ranks, plans, summarizes, drafts; typed application services own permissions, risk, budgets, and mutations.

Enterprise Control Points

- Tenant/project/environment scope on every row; classification, purpose, and residency filtered before retrieval and again before prompts.
- Operator-facing claims instantiate from a digest-pinned template registry; a failed predicate is withheld, never softened into prose.
- Immutable histories with superseding records; dual-written receipts; independent verification shares nothing with the producer it judges.
- Preview platform features carry tested degradation paths and are never the sole control on a consequential action.

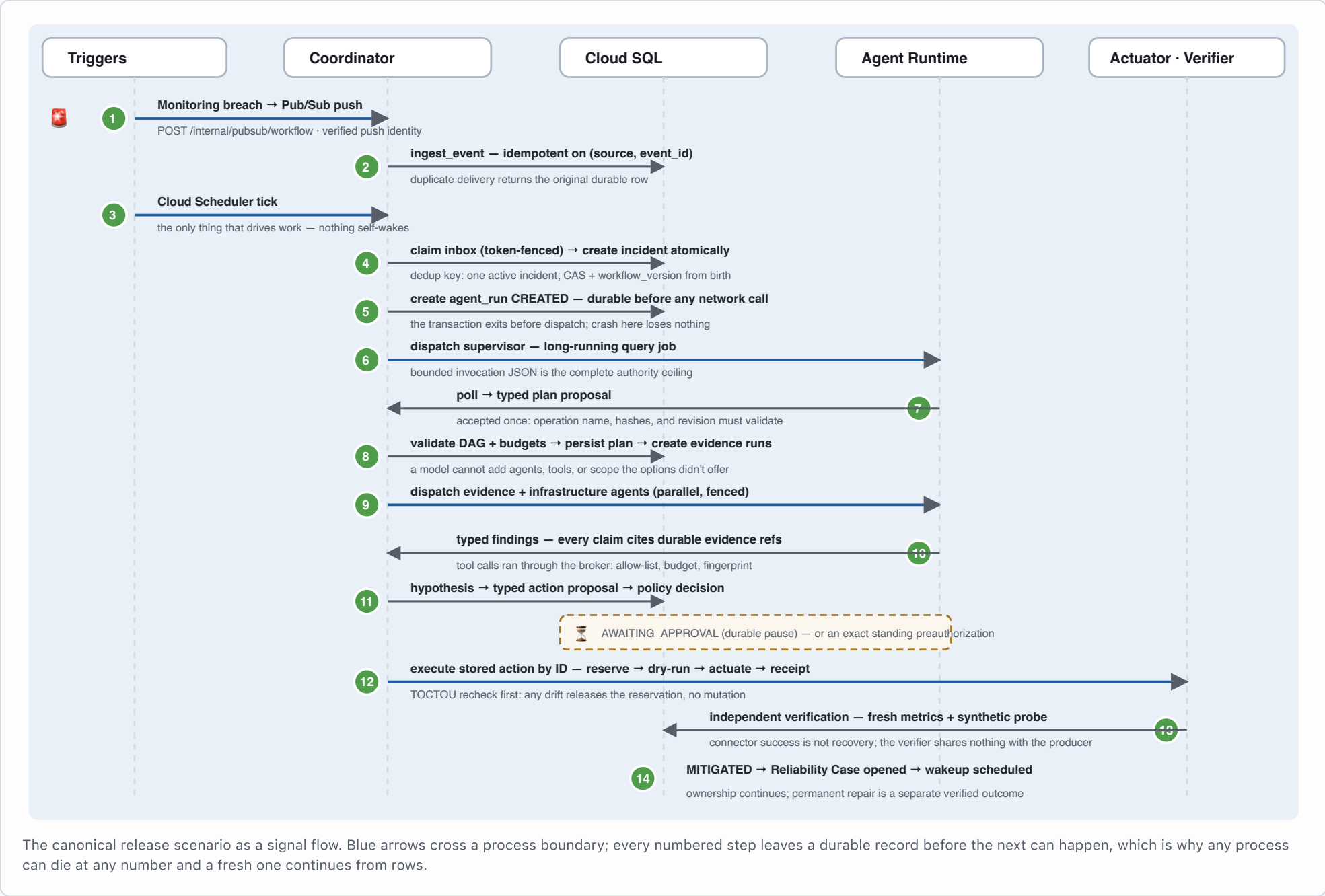
02 One Turn Through The System

Google's ADK lab draws one message moving through the Runner. This is the same drawing for Solvan: one alert becoming one dispatched, validated, human-approved, verified turn. The durable write is drawn where it happens — **before** dispatch, not after.



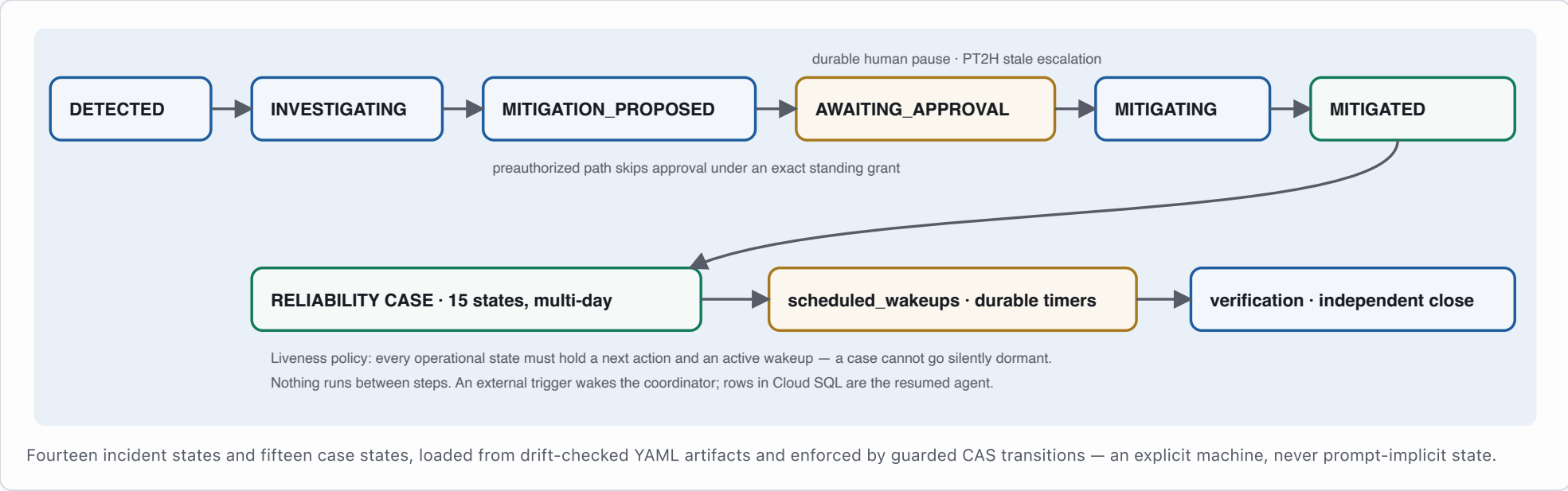
Solvan running the ADK lab's Runner loop at production discipline: the alert speaks (1), the durable row exists before any network call (2), the invocation JSON is the whole authority ceiling (3), the planner plans with zero tools (4), evidence tools run behind the broker (5), the pause is cryptographic (6), and execution is verified by a stranger (7). Evidence: `test_coordinator_commits_incident_and_inbox_completion_atomically`, the contracts in `src/solvan/agents/contracts.py`, and the swimlane + tables on the following pages.

03 Signal Flow: Alert To Verified Recovery



The canonical release scenario as a signal flow. Blue arrows cross a process boundary; every numbered step leaves a durable record before the next can happen, which is why any process can die at any number and a fresh one continues from rows.

04 Incident Lifecycle And Durable State



Durability mechanics — each proven by a named test

Mechanism	Guarantee	Verify
Token-fenced claims (inbox, outbox, wakeups)	A crashed worker's claim expires and is recovered; the stale token can no longer complete the work.	test_expired_inbox_claim_is_recovered_and_old_token_is_fenced
Restart recovery without sessions	A fresh process resumes a dispatched runtime operation from rows alone — no session, no memory of the dead process.	test_dispatched_runtime_operation_is_recovered_by_fresh_process_without_session_state
Poison-event quarantine	An event that crash-loops its handler is parked durably after a bounded claim budget (5 inbox / 25 outbox); it can never occupy a worker again.	test_crash_looping_inbox_event_is_quarantined_after_bounded_claims
Durable retry backoff	A failed dispatch defers its single declared fallback behind retry_not_before ; a transient fault cannot burn the final attempt instantly, across restarts.	test_declared_agent_fallback_is_attempted_once_after_dispatch_failure
Concurrency at scale	Twenty-five simultaneous incidents are neither lost nor duplicated under contention.	test_twenty_five_simultaneous_incidents_are_not_lost_or_duplicated
Atomic coordination	Incident creation and inbox completion commit together or not at all; duplicate ingress returns the original durable event.	test_coordinator_commits_incident_and_inbox_completion_atomically

Why rows, not resumed conversations

The ADK lab's own ResumabilityConfig note warns: "any temporary / in-memory state will be lost upon resumption." Solvan takes the warning to its conclusion: a half-finished model invocation is never trustworthy state. The unit of recovery is the durable agent_run ; recovery is a fresh, fenced attempt rebuilt from authoritative rows, and idempotency constraints make repeated effects impossible rather than unlikely.

Suspension is a state, not a parked process

AWAITING_APPROVAL and scheduled_wakeups are Cloud SQL states with fenced claims and expiry — an 18-hour human pause or a 7-day case costs zero compute and survives any deployment. The stale-progress policy escalates abandoned work instead of letting it rot; case liveness rules make silent dormancy a schema violation.

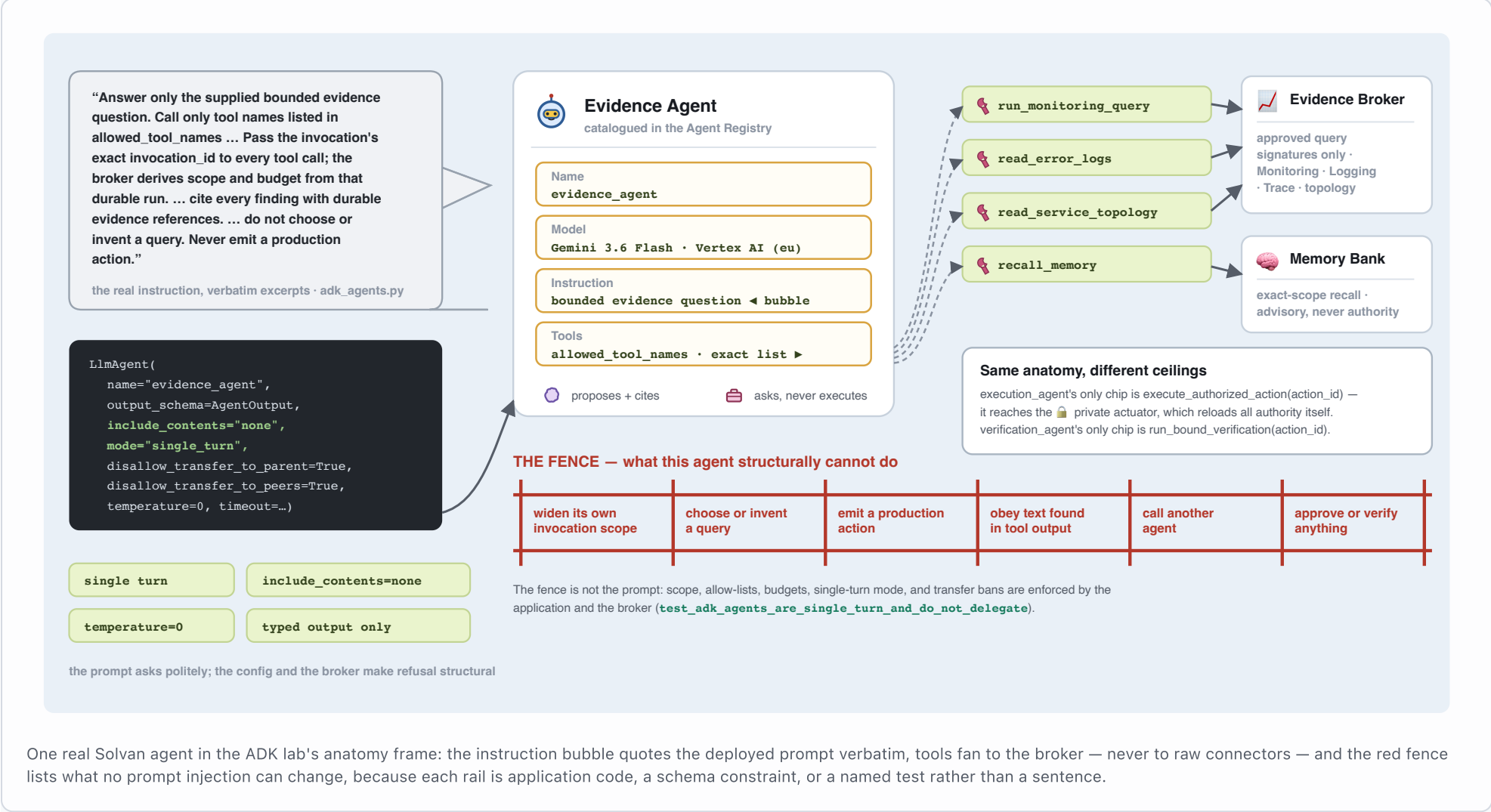
05 Signal Flow: Kill The Process, Keep The Incident

Each row kills the process at a different point in the pipeline. The left box is the failure; the right box is the deterministic mechanism that recovers it — no operator action, no lost or duplicated work. Every row is a named test in the merge gate.



06 Agent Anatomy: One Bounded Institutional Agent

The ADK lab teaches that an agent "does not run itself — it is a description: a name, a model, an instruction, a list of tools." Here is one real Solvan description — the Evidence Agent as built in `src/solvan/agents/adk_agents.py` — with the instruction it actually carries and the fence around everything it cannot do.



One real Solvan agent in the ADK lab's anatomy frame: the instruction bubble quotes the deployed prompt verbatim, tools fan to the broker — never to raw connectors — and the red fence lists what no prompt injection can change, because each rail is application code, a schema constraint, or a named test rather than a sentence.

07 Agent Plane And Model Governance

Agent	Authority ceiling	Cannot
incident-supervisor	Propose an investigation plan from application-supplied agent and scope options.	Call tools, call agents, propose actions or transitions.
evidence-agent	Answer one bounded evidence question via allow-listed read tools; cite durable references.	Choose queries outside the plan, emit actions, touch production.
infrastructure-agent	Inspect registered read-only deployment/topology/capacity tools against the frozen snapshot.	Deploy, restart, roll back, scale, or open data sessions.
execution-agent	Request the stored, approved action by ID through the private actuator.	Author payloads, select targets, approve, or verify itself.
verification-agent	Narrate the application-computed verdict from fresh telemetry and probes.	Choose the profile, weaken criteria, or share context with the producer.
workspace-agent	Lead investigation and implement repair inside a governed workspace.	Self-confirm, approve, merge, deploy, verify, resolve, or promote its own output.

Bounded invocation

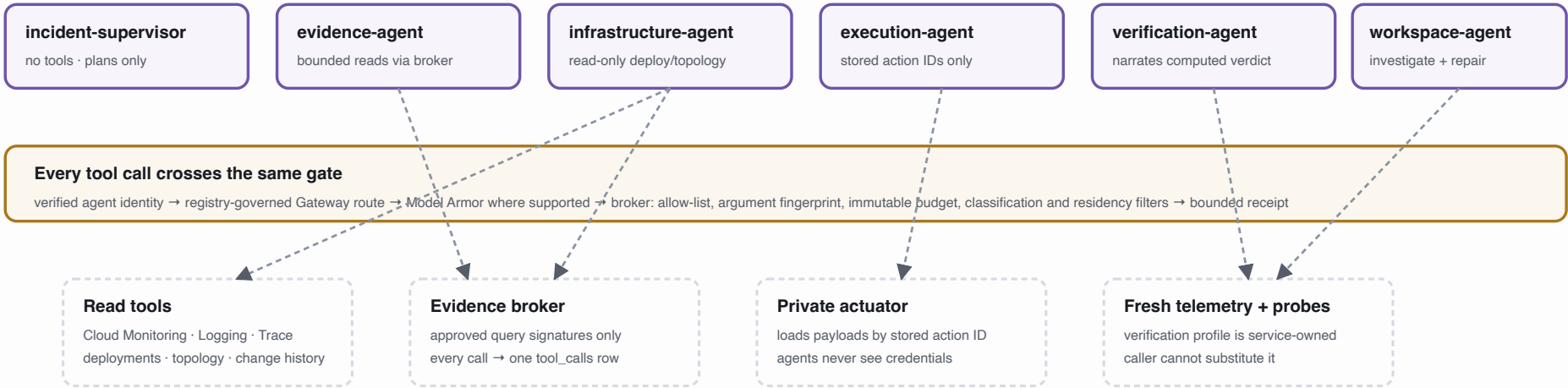
- Invocation JSON is the complete authority ceiling: scope, tools, deadline, budget, evidence refs.
- Pinned `input_hash / output_hash` ; single-turn; `include_contents="none"` isolation.
- Deterministic loop enforcement outside the model: fingerprinted calls, progress checks, budget stops.

Untrusted by default

- Logs, traces, repos, tickets, tool responses, model output, and Memory Bank content are data, never instructions.
- Model Armor screens supported paths inline; verdicts persist on evidence and gate memory promotion (`armor_verdict_ref` NOT NULL).
- Injection demo: a malicious log line stays evidence and cannot become instruction or memory.

Claims, not prose authority

- Operator-facing factual claims instantiate from an enumerated, digest-pinned template registry.
- The application derives kind, polarity, and sentence, and verifies the predicate in code against cited records.
- A failed predicate is withheld or replaced by its holding form — never softened into fluent text.

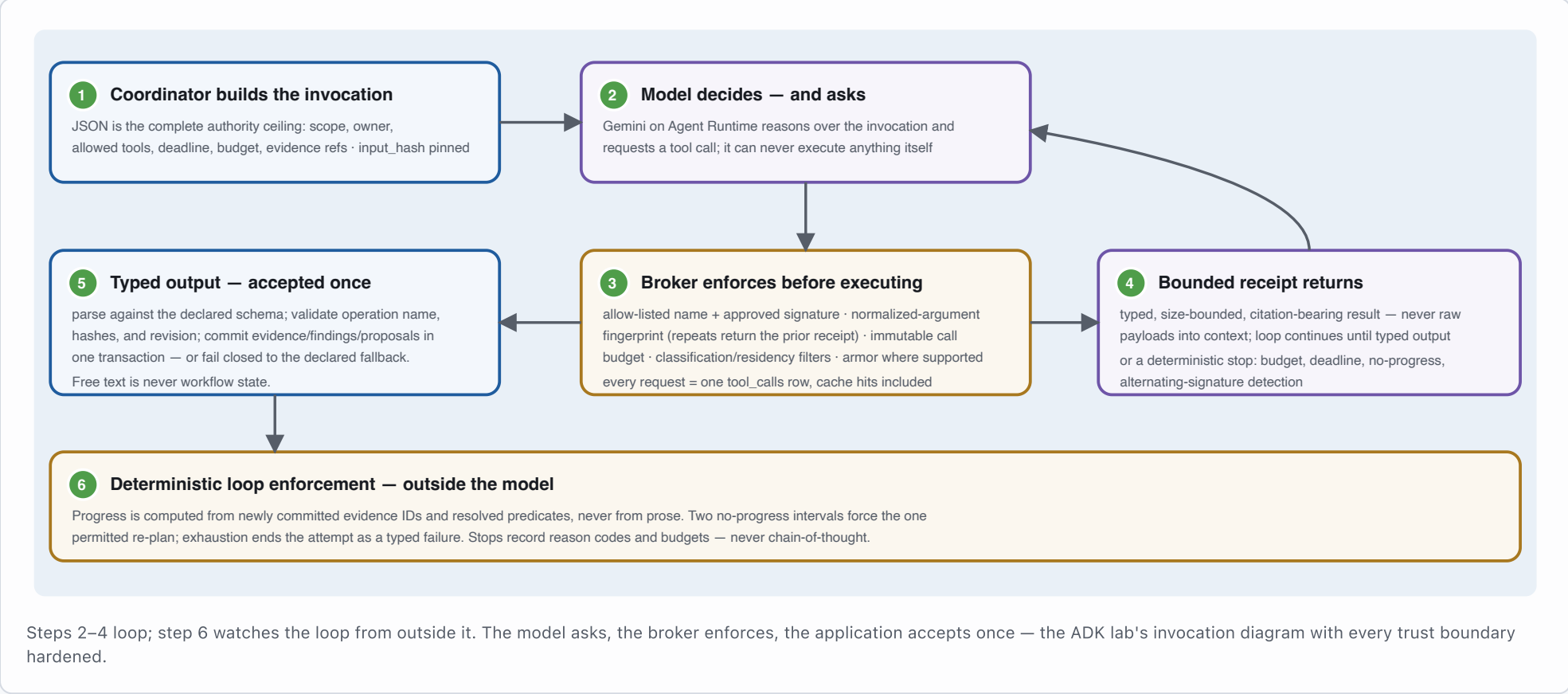


The fleet graph, Solvan-style: agents on top, tool surfaces below, and one amber gate every call must cross. The supervisor deliberately has no edge to any tool — a planner that cannot touch the world cannot be prompt-injected into touching it.

Memory is earned, not accumulated. Memory Bank recall is exact-scope and non-authoritative. Promotion into durable memory requires type-specific confirmation rules plus a recorded armor verdict; unconfirmed or poisoned candidates are quarantined (`test_unconfirmed_or_poisoned_candidate_is_quarantined` , `test_memory_candidate_promotes_once_and_retry_returns_durable_receipt`).

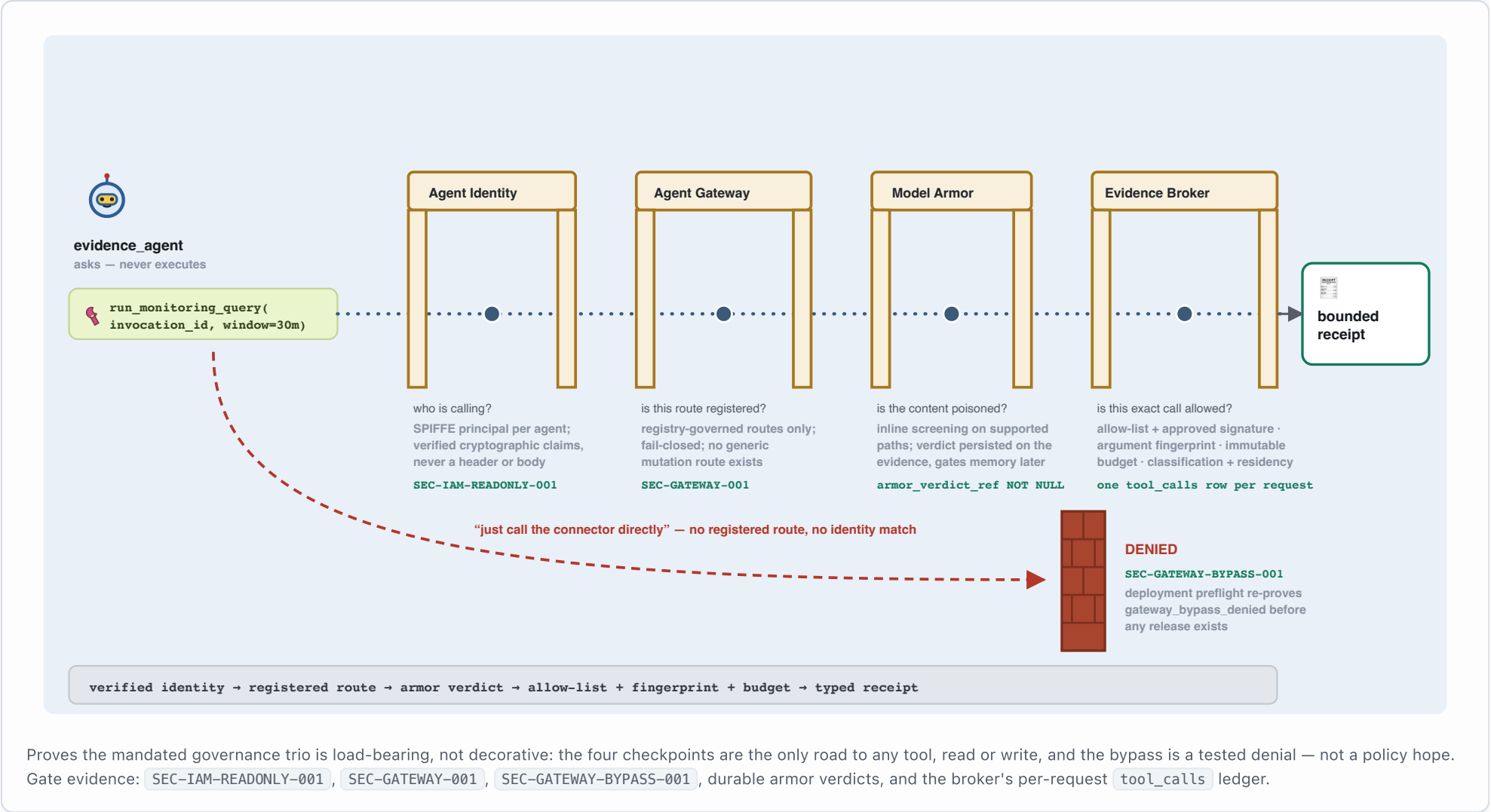
08 Signal Flow: The Bounded Invocation Loop

The ADK lab teaches that "the model never runs your code — it asks." Solvan keeps that loop and wraps every side of it in deterministic enforcement. One invocation, numbered:



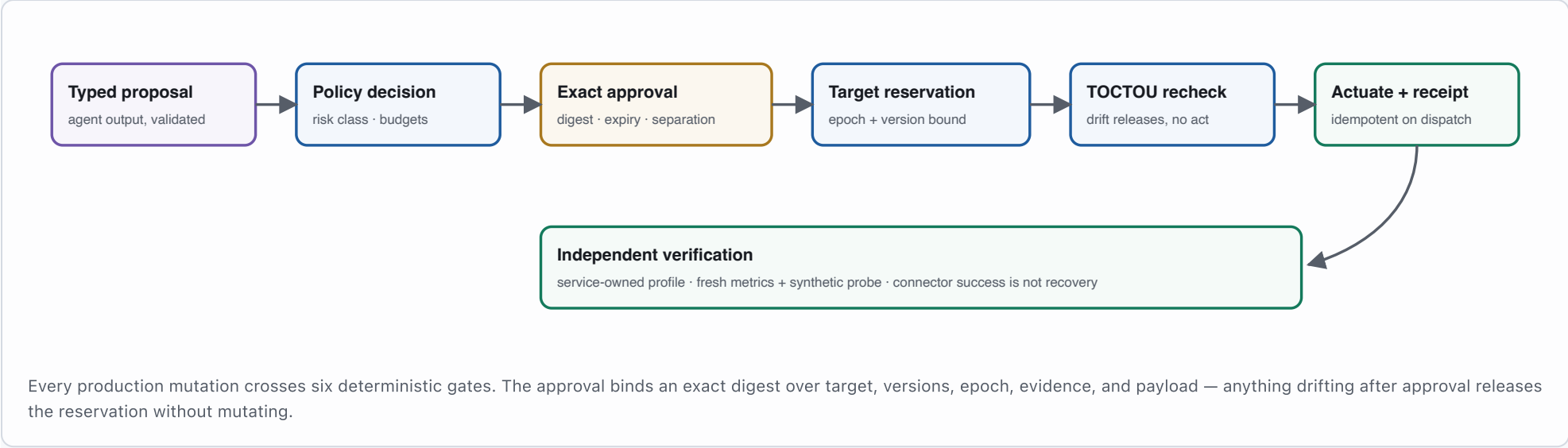
09 The Governance Gauntlet: One Tool Call, Four Gates

Step 3 of the invocation loop, magnified. Every tool call any agent makes walks the same four physical checkpoints — and the only other road ends at a wall with a test name on it.



Proves the mandated governance trio is load-bearing, not decorative: the four checkpoints are the only road to any tool, read or write, and the bypass is a tested denial — not a policy hope. Gate evidence: `SEC-IAM-READONLY-001` , `SEC-GATEWAY-001` , `SEC-GATEWAY-BYPASS-001` , durable armor verdicts, and the broker's per-request `tool_calls` ledger.

10 Action Safety And Verification

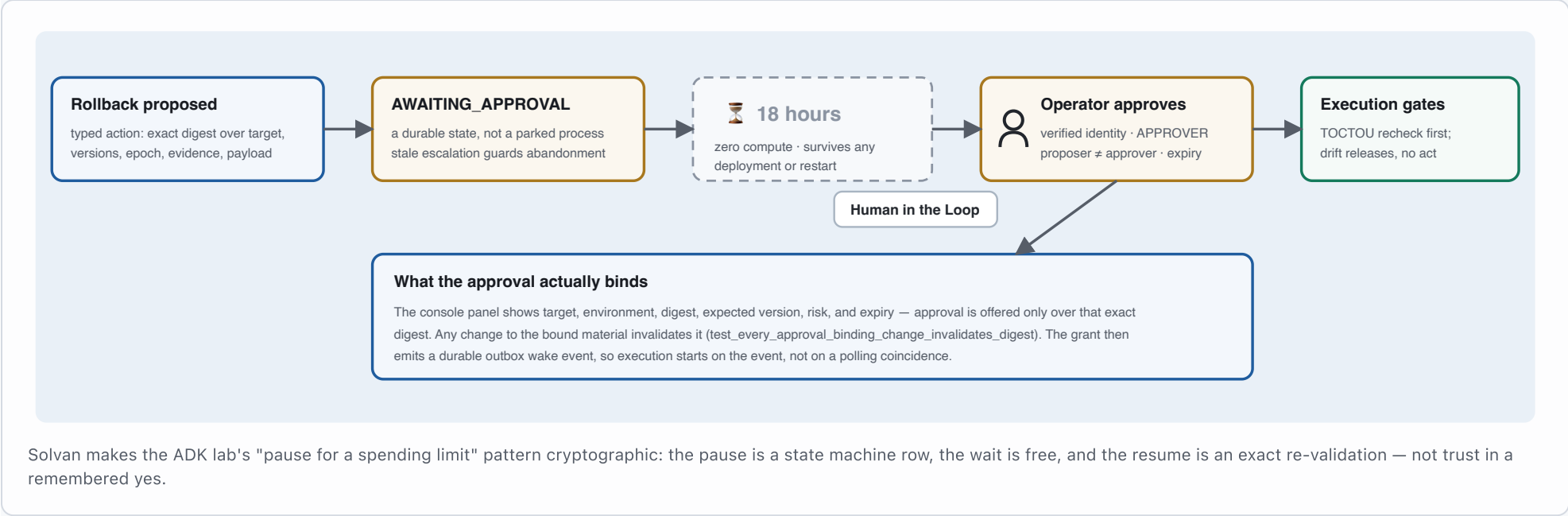


Idempotency and staleness — layered, schema-enforced

Layer	Constraint	Verify
Alert ingress	Unique (source, source_event_id) ; duplicates return the original event.	test_duplicate_ingress_returns_the_same_durable_event
Agent attempts	Partial unique index: one active attempt per logical step; unique invocation_id and provider_request_id .	agent_runs_one_active_attempt (DDL)
Approvals	Exact digest over reviewable material; proposer ≠ approver in code and SQL; expiry; stale approval rejected.	test_every_approval_binding_change_invalidates_digest
Execution	Re-check state/version/policy at execution; changed target releases without mutation.	test_execution_recheck_rejects_toctou_changes , test_changed_connector_target_releases_without_mutation
Receipts	Unique per dispatch (INV-T-08); effect-compared before reconcile; undo derived from observed pre-state.	actuator DDL checks
Repetition	Action budgets, repeated-action limits, cooldown — flapping is structurally bounded.	test_action_budget_enforces_every_deterministic_boundary

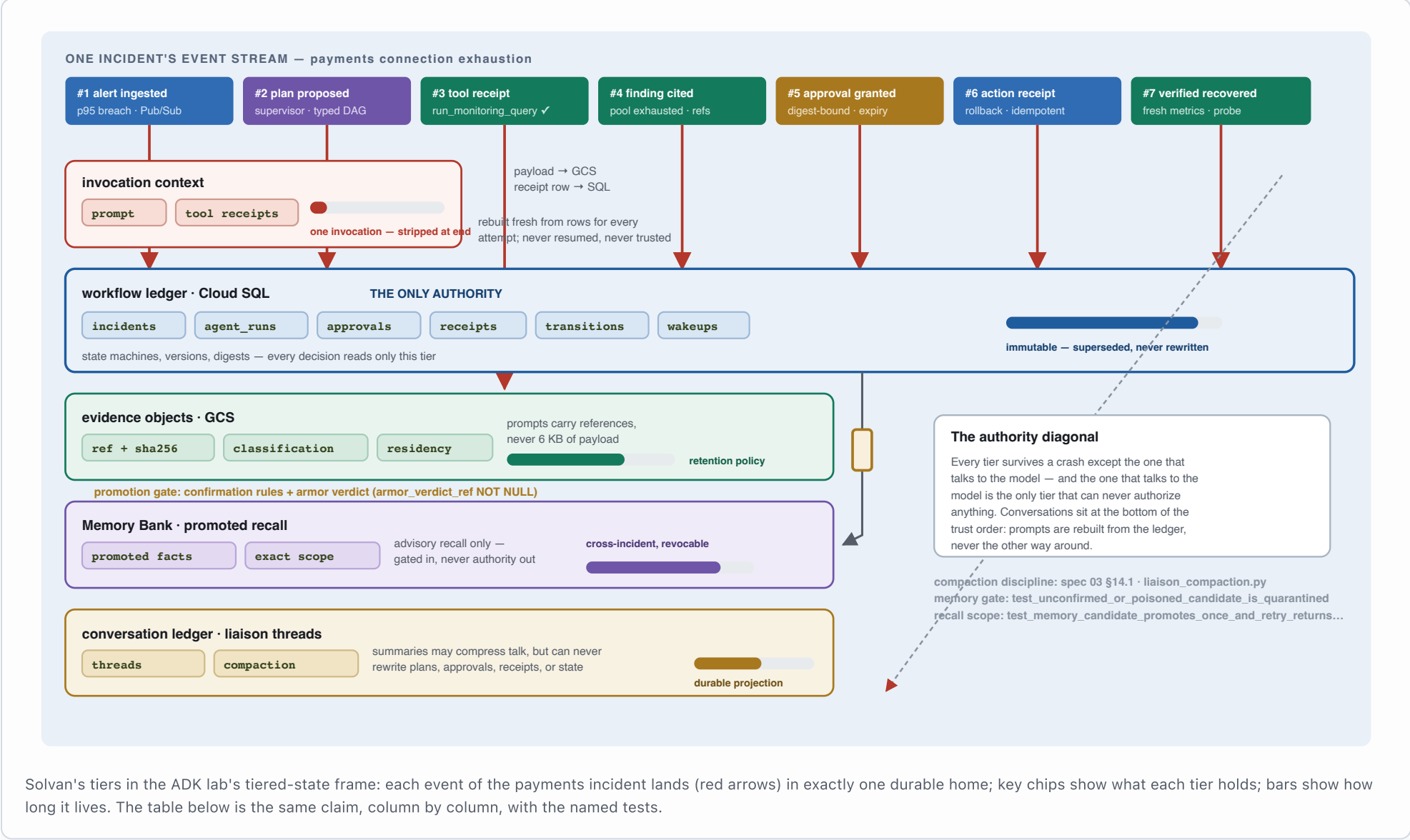
Verification independence. The verifier never shares identity, provider process, conversation, or mutable artifacts with the producer it adjudicates; the remediation planner cannot choose a weaker success criterion; a caller cannot substitute the verification profile (test_caller_cannot_substitute_verification_profile).

11 Signal Flow: A Human Pause That Costs Nothing



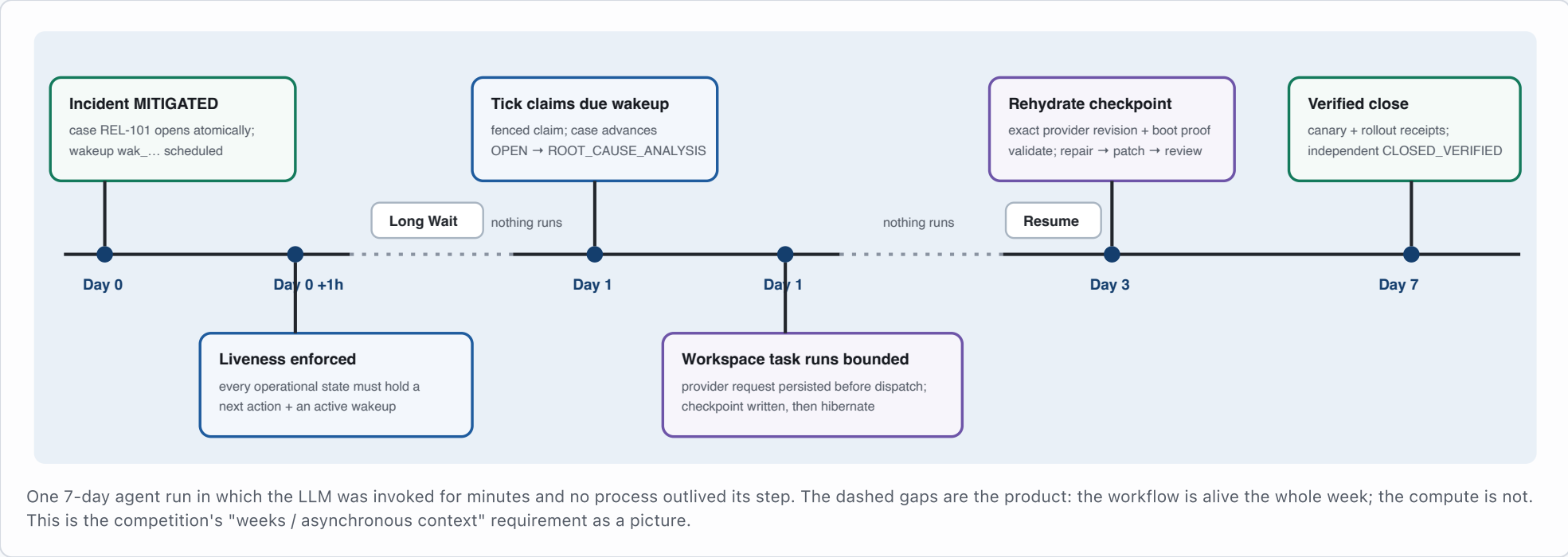
12 Managing State, Solvan Edition

The ADK lab's "Managing Context" figure asks one question of every fact: where does it live, and how long does that place last? Here is one real incident's event stream, with a red arrow from each event to the tier where that fact durably lands — and the authority diagonal that makes the whole security argument.



Tier	Lifetime	Survives crash?	Survives deploy?	Survives incident close?	Can carry authority?
invocation context prompt + tool receipts in one bounded run	one invocation	no	no	no	never
agent_runs + steps attempts, hashes, budgets, traces	incident lifetime + audit	yes	yes	retained	no — records of work, not authority
evidence objects (GCS) hash-pinned, classified, redacted	retention policy	yes	yes	yes	citable, never instructions
workflow ledger incidents, cases, actions, approvals, receipts, transitions	immutable — superseded, never rewritten	yes	yes	yes	the only authority
Memory Bank promoted, scoped recall	cross-incident, revocable	yes	yes	yes	no — gated in, advisory out
conversation ledger liaison threads, compaction summaries	durable projection	yes	yes	yes	no — reads state, never mints it

13 Signal Flow: Seven Days, Zero Resident Processes



Where the ADK lab ends and Solvan begins. The ADK lab parks one invocation in one queue and resumes it. Solvan holds an open-ended case across many bounded runs, two state machines, a workspace with hibernated checkpoints, and a liveness policy that makes silent dormancy a schema violation — the same doctrine at institutional scale.

14 Requirements Scoreboard: Solvan Through The Lens Of The ADK Lab

Every mandated technology, and every pattern taught by the ADK lab (adk2003), as a scoreboard: green check = implemented and proven by the named pointer; amber ▲ = Solvan goes measurably beyond the lab baseline there. The two tables on the next page carry the full evidence rows.

THE TWELVE MANDATED PROOFS

= beyond the lab baseline

Gemini 3.5+ Gemini 3.6 Flash pinned in every Runtime deployment; visible in traces + README <code>model resource · eu endpoint</code> optional 3.1 Pro Preview deep workspace is an explicit global-location exception	Google ADK ADK 2 agent source and manifests, checked in CI; single-turn, schema-bound <code>src/solván/agents/adk_agents.py</code>	Google Cloud infra Cloud Run · Cloud SQL · Pub/Sub · Scheduler · GCS — load-bearing, never décor <code>tools/deploy_release.py</code>	Agent Registry live catalog: versions, capabilities, departments; drives the Gateway <code>console Fleet tab</code>
Agent Runtime real bounded executions; the lab resumes one parked invocation — Solván re-arms from rows <code>test_dispatched_runtime_operation...</code>	Memory Bank exact-scope recall; promotion and denial both demonstrated live, armor-gated <code>test_memory_candidate_promotes_once...</code>	Agent Identity distinct SPIFFE principal per agent; least-privilege IAM proven by tests <code>SEC-IAM-READONLY-001</code>	Agent Gateway live allowed route plus denied bypass; fail-closed by construction <code>SEC-GATEWAY-BYPASS-001</code>
Model Armor injection verdict recorded while the investigation continues safely <code>durable verdicts on evidence + memory</code>	Agent Observability OTel trace DAG correlated to the domain timeline; decisions, never chain-of-thought <code>trace_id / span_id on every run</code>	Weeks · async context a persisted multi-day case with live next-step resume — zero resident processes all week <code>test_mitigated_incident_opens_one_case...</code>	Production-data safety classification + residency policy before retrieval and again before prompts <code>Alpha SDK: PUBLIC + synthetic only</code>

THE ADK LAB'S TAUGHT PATTERNS (adk2003), ENFORCED AS CONSTRAINTS

✓ events, not prompts spec 01 §2.1	✓ trigger / triggerer split CT-INBOX-001	✓ durable state restart-recovery test	✓ tiered memory + artifacts memory gating tests	✓ compaction distrust spec 03 §14.1	✓ sub-agent isolation ▲ coordinator-only dispatch
✓ suspension / resume ▲ states, not parked processes	✓ staleness guards TOCTOU recheck tests	✓ idempotency keys six schema-enforced layers	✓ human-in-the-loop digest-bound approvals	✓ workflow graphs validate_investigation_plan	✓ cloud deployment ▲ + the governance trio on top

▲ divergences are argued from the ADK lab's own caveats — no mid-invocation resume, no prompt-side orchestration, sessions never authority (next page).

The submission checklist's architecture-diagram requirement, scored: twelve mandated proofs and twelve ADK-lab patterns, each with its enforcing mechanism and pointer. Amber ▲ marks where the ADK lab's habit became a Solvan constraint that goes further than the lab asked.

15 The Lens In Detail: Requirement, Mechanism, Proof

Mandatory technology proof (rule/track requirements)

Requirement	Where Solvan proves it
Gemini 3.5+	Model resource pinned in every Runtime deployment; visible in traces and README.
Google agent framework	Google ADK 2 agent source (<code>src/solvan/agents/adk_agents.py</code>) and manifests, checked in CI.
Google Cloud infrastructure	Agent Runtime, Cloud Run, Cloud SQL, Pub/Sub, Cloud Scheduler, GCS — all load-bearing, none decorative.
Agent Registry	Live catalog with versions, capabilities, departments; console Fleet tab renders it.
Agent Runtime	Real bounded executions via long-running query jobs; crash/resume proven by the restart-recovery test.
Memory Bank	Exact-scope recall; gated promotion and denial demonstrated live.
Agent Identity	Distinct SPIFFE principal per agent; least-privilege IAM matrix proven by <code>SEC-IAM-READONLY-001</code> , <code>SEC-ACTUATOR-IAM-001</code> .
Agent Gateway	Live allowed route plus denied bypass (<code>SEC-GATEWAY-001</code> , <code>SEC-GATEWAY-BYPASS-001</code>); preflight checks <code>gateway_bypass_denied</code> .
Model Armor	Injection verdict recorded while investigation continues safely; verdicts durable on evidence and memory.
Agent Observability	OTel trace DAG correlated to the domain timeline through <code>trace_id</code> / <code>span_id</code> on every run.
Weeks / asynchronous context	Persisted multi-day Reliability Case with live next-step resume from a durable wakeup.
Production-data safety	Real synthetic GCP telemetry under classification/residency policy; the Alpha SDK provider receives only attested PUBLIC + synthetic inputs.

The ADK lab's patterns (adk2003), taken to production

Lab lesson	Solvan enforcement	Verify
Prompts can't start themselves	Founding principle: incidents begin from events, not prompts.	spec 01 §2.1
Trigger / triggerer split	Scheduler → tick endpoints; Pub/Sub push → fenced inbox; being callable ≠ being called.	<code>apps/coordinator/main.py</code>
Durable state storage	Cloud SQL as authoritative workflow ledger — not a session cache.	restart-recovery test
Tiered memory + artifacts	Run context → workflow rows → gated Memory Bank; GCS artifacts by <code>ref + sha256</code> .	memory gating tests
Compaction distrust	Invariant: compaction can never rewrite plans, evidence, approvals, receipts, or state; prompts rebuild from records.	spec 03 §14.1
Sub-agent isolation	<code>include_contents="none"</code> plus structural isolation: coordinator-only dispatch.	<code>test_adk_agents_are_single_turn_and_do_not_delegate</code>
Suspension / resumption	Durable pause states + fresh fenced attempts instead of resuming conversations (see 05).	wakeup + recovery tests
Staleness guards	Target epochs, digest-bound approvals, TOCTOU rechecks at execution.	TOCTOU tests (10)
Idempotency keys	Six schema-enforced layers; the double-purchase bug is impossible by constraint.	idempotency table (10)
Human-in-the-loop	Approval as a cryptographic act, not a stored preference.	approval tests (10, 11)
Workflow graphs	Model proposes the DAG; application validates; coordinator executes with fencing and one declared fallback.	<code>validate_investigation_plan</code>
Cloud deployment + Trace	The ADK lab's full deployment stack plus the governance trio on top.	<code>tools/deploy_release.py</code> , preflight

Deliberate divergences — argued from the ADK lab's own caveats. No mid-invocation resume (the lab: temp state "will be lost upon resumption" — so Solvan never trusts a parked invocation). No prompt-side orchestration (the lab isolates its sub-agent from context; Solvan makes isolation structural). Sessions are never authority (the lab spends two tasks on why transcripts and summaries mislead; Solvan demotes conversations to projections). Each divergence finishes an argument the lab starts.

16 Data, Memory, And Conversational Surface

Release schema · 61 tables

- Incidents, cases, plans, steps, runs, tool calls, evidence, hypotheses, actions, approvals, reservations, receipts, transitions, wakeups, inbox/outbox, memory, workspaces.
- Loaded from empty into PostgreSQL 16 in CI; constraint oracles assert negative cases; append-only audit triggers.

Operability schema · 54 tables

- Governed Tool Catalog and capability profiles; Operational Guidance revisions with registered completion predicates; trigger policies; skills interchange records.
- Status: **implemented and locally verified, beyond the lens** (specs 16–18) — guidance selection, tool-catalog wiring, and quarantined skills import run in real coordinator paths; cloud release qualification is deliberately not yet claimed.

Conversation schema · 34 tables

- One canonical Liaison ledger across console and channels: read/steer grants, per-part access modes (empty set denies), durable delivery, compaction.
- Identity from verified claims only; a read grant never proves write or steer authority.
- Status: **implemented and locally verified, beyond the lens** (spec 14) — five channel liaisons with the Slack lifecycle, steer confirmation, subscriptions, and deterministic compaction.

Further target schemas beyond these two — alert triage (28 tables, implemented), outcome quality (17), production graph (16), connection scope (4), relay (18), SaaS scale (28) — bring the platform's schema surface to 260 tables.

Evidence with provenance

- Every evidence item: source, window, query, hash, classification, residency, retrieval actor, freshness expiry, armor verdict.
- Citations must resolve to stored records; an unresolvable reference renders inert and demotes the statement to inferred.
- Validated vs inferred findings are visually and semantically distinct in the console.

Conversation reads, never mints

- Chat and channel liaisons read reader-filtered projections of durable state; asking a question can never create a fact.
- Approval by conversation surfaces the same digest-bound panel — inline text never becomes an approval.
- Every durable conversational part declares an access mode; visibility is never granted by omission.

Skills without surrendering authority (target contract). The open Agent Skills format is adopted as a quarantined interchange profile: packages are hardened, scanned, and held as inert records; a human compiles them into governed guidance through evaluation and independent approval; scripts never execute; `allowed-tools` grants nothing; activation pins content and never fetches origins. Four adversarial review passes; 24 acceptance tests specified.

17 Enterprise Platform: Shown, Shipped, Specified

The honest enterprise claim, in three levels. What the lens demo *shows* is release-gated. What the submission *ships* goes further: most of the enterprise expansion is implemented and locally verified in this same codebase — it simply is not part of the lens demonstration, and its cloud release qualification is not yet claimed. What remains is *specified*: reviewed normative contracts with acceptance tests, not roadmap prose. Solvan is built for this competition and continues beyond it as Ruhu Inc.'s production reliability platform.

Shown — release-gated, through the lens

- Tenant/project/environment scope on every row; region-pinned deployment with release-blocking policy checks.
- Per-agent SPIFFE identity; least-privilege IAM proven by tests; read-only agents cannot reach mutation endpoints.
- Registry-governed, fail-closed Gateway with no generic mutation route; bypass denial tested.
- Model Armor inline on supported paths with durable verdicts; memory promotion gated on them.
- Immutable histories, superseding records, dual-written receipts, independent verification.
- Ten-stage merge gate: lint, types, architecture ceilings, docs, executable schema contracts, 953 tests, safety evals, console build, browser critical path.
- PR-001..050 requirement-to-test traceability; CI fails on a missing or skipped required test.

Shipped — implemented beyond the lens

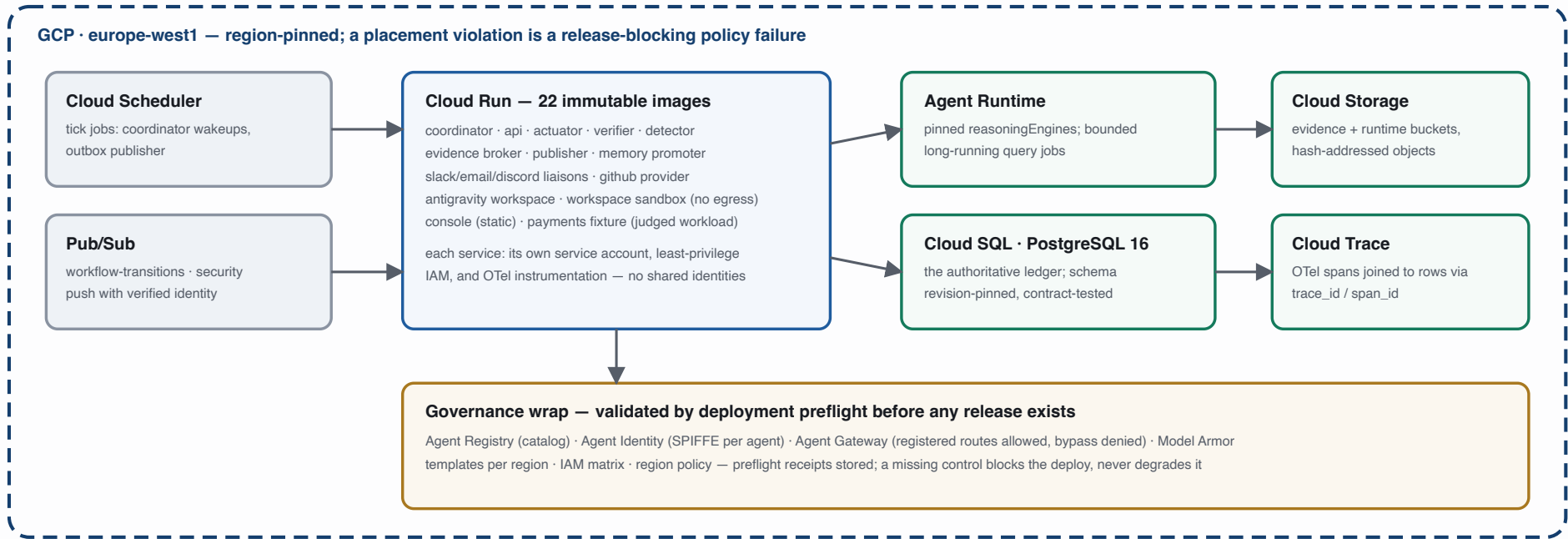
- **Conversational surface** (spec 14): five channel liaisons on one ledger — Slack lifecycle, steer confirmation, subscriptions, compaction; locally verified.
- **Governed Tool Catalog** (16) and **Operational Guidance** (17): wired into real coordinator dispatch with hash-bound selection and computed completion.
- **Agent Skills interchange** (18): quarantined import implemented; local implementation gate passed.
- **Alert triage** (21): implemented and locally verified — 28-table schema, fenced-claim runtime, console surfaces.
- **Production environment model** (20): production graph and earned-autonomy receipts substantially implemented.
- All of the above ship in the submitted repository; none is used as lens evidence, and cloud release qualification is not claimed.

Specified — reviewed target contracts

- **SaaS scale and isolation** (19): cell contract, content-free routing, placement grants — implementation under way.
- **Solvant Relay** (22): customer-resident read-only evidence collection; specification complete, implementation early.
- **Customer-deployed actuator** (13): Solvan never holds a credential capable of mutating a customer estate; kill switch and rate budget customer-owned.
- **Multi-tenant sovereignty**: per-tenant CMEK across a tenant population (depends on 19).
- **Ruhu adoption profile** (11): observe-only → approval-bound rollback → bounded autonomy, each phase gated by receipts.

Enterprise property	Mechanism	Status
Data sovereignty	Classification + residency filters before retrieval and before prompts; CMEK-bound storage for tenant material.	implemented (judged scope) / target (multi-tenant)
Credential posture	Every stored credential records posture; federated short-lived preferred; long-lived keys read-only under CMEK, surfaced in console.	target (spec 13)
Auditability	Append-only transitions and audit events; structured decisions; no chain-of-thought recorded anywhere.	implemented
Fail-closed configuration	No config value may downgrade identity, policy, or isolation; absent or unparsable policy refuses.	implemented (invariant + tests)
Preview-feature honesty	Launch stages recorded with retrieval dates; preview features carry tested degradation paths.	implemented (source register + preflight)

18 Deployment, Quality Gate, And Scoring Alignment



The ADK lab's step-10 topology — Cloud Run + Scheduler + Pub/Sub + Cloud SQL + GCS + Trace — with Agent Runtime carrying the model work and the governance plane wrapped around all of it.

Deployment topology

- **Cloud Run:** coordinator, API, actuator, verifier, detector, evidence broker, publisher, liaisons, workspace services — each with its own service identity.
- **Agent Runtime:** bounded agent executions as long-running query jobs against pinned `reasoningEngines` resources.
- **Cloud SQL (PostgreSQL 16):** the authoritative ledger, region-pinned to `eu-west1`.
- **Pub/Sub + Cloud Scheduler:** the only things that wake anything; push endpoints ingest, ticks drive.
- **GCS:** evidence and runtime objects, hash-addressed; **Cloud Trace:** OTEL spans correlated to domain rows.
- **Cloud Build:** 22 immutable image outputs; deployment preflight validates agents, identities, gateway policies, and armor templates before release.

The ten-stage merge gate (scripts/check)

1. Python lint and format
2. Python types (mypy, 315 files)
3. Architecture boundaries, agent manifests, Ruhu profile, Cloud Build contract
4. Documentation and repository knowledge
5. Executable schema + transition contracts on clean PostgreSQL 16
6. Unit and harness tests with coverage
7. Agent safety and trajectory evaluation
8. Console types
9. Console production build
10. Browser critical path

Local evidence never counts as a GCP release receipt; cloud proofs are captured separately at deploy time.

Scoring alignment

Axis	Weight	Solvan's story
Innovation and operational utility	40%	The two-plane authority split; verification independence; claims from digest-pinned templates; autonomy as an earned receipt; a conversational surface that cannot mint facts. Utility is the demo itself: a real induced degradation detected, mitigated, verified, and owned to permanent repair without a prompt.
Architectural discipline and tech stack	30%	Section 14's scoreboard and Section 15's tables are the discipline story told in the assessors' own vocabulary, with the mandated stack used as enforcement planes, not checkboxes. Every arrow in Section 01 has a test behind it.
Demo and production readiness	30%	Kill a process mid-incident on camera and watch rows resume it — that is a named required test, not a rehearsed trick. Judges can run the same gate we run.

Verify it yourself in ten minutes

```
scripts/bootstrap    # locked dependencies and Chromium
scripts/start        # isolated stack: PostgreSQL, API, console, logs, traces
scripts/check        # the full ten-stage gate, including restart-recovery,
                     # idempotency, TOCTOU, quarantine, and approval-digest tests
```

Then open the console: approvals show target, environment, digest, version, risk, and expiry; evidence shows sources, freshness, classification, and armor verdicts; the fleet screen shows discovery, effective policy provenance, and enforced permission. Control made visible is the product thesis — and the judging interface.

19 The Architecture Beat

The one screen the four-minute video holds while the narrator says what Solvan is: the poster, then the scoreboard. Alert to verified close along the dotted path; every mandated pillar as a product card; twelve checks and the three amber exceedances.

